The Open Group Standard

Open Messaging Interface (O-MI), The Open Group Standard for the Internet of Things (IoT), Version 2.0

Copyright © 2019, The Open Group

The Open Group hereby authorizes you to use this document for any purpose, PROVIDED THAT any copy of this document, or any part thereof, which you make shall retain all copyright and other proprietary notices contained herein.

This document may contain other proprietary notices and copyright information.

Nothing contained herein shall be construed as conferring by implication, estoppel, or otherwise any license or right under any patent or trademark of The Open Group or any third party. Except as expressly provided above, nothing contained herein shall be construed as conferring any license or right under any copyright of The Open Group.

Note that any product, process, or technology in this document may be the subject of other intellectual property rights reserved by The Open Group, and may not be licensed hereunder.

This document is provided "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. Some jurisdictions do not allow the exclusion of implied warranties, so the above exclusion may not apply to you.

Any publication of The Open Group may include technical inaccuracies or typographical errors. Changes may be periodically made to these publications; these changes will be incorporated in new editions of these publications. The Open Group may make improvements and/or changes in the products and/or the programs described in these publications at any time without notice.

Should any viewer of this document respond with information including feedback data, such as questions, comments, suggestions, or the like regarding the content of this document, such information shall be deemed to be non-confidential and The Open Group shall have no obligation of any kind with respect to such information and shall be free to reproduce, use, disclose, and distribute the information to others without limitation. Further, The Open Group shall be free to use any ideas, concepts, know-how, or techniques contained in such information for any purpose whatsoever including but not limited to developing, manufacturing, and marketing products incorporating such information.

If you did not obtain this copy through The Open Group, it may not be the latest version. For your convenience, the latest version of this publication may be downloaded at www.opengroup.org/library.

The Open Group Standard

Open Messaging Interface (O-MI), The Open Group Standard for the Internet of Things (IoT), Version 2.0

ISBN: 1-947754-42-3

Document Number: C19E

Published by The Open Group, December 2019.

Comments relating to the material contained in this document may be submitted to:

The Open Group, Apex Plaza, Forbury Road, Reading, Berkshire, RG1 1AX, United Kingdom

or by electronic mail to:

ogspecs@opengroup.org

Contents

1 Introduction 1
- 1.1 Objective 1
- 1.2 Overview 1
- 1.3 Conformance 2
- 1.4 Normative References 3
- 1.5 Terminology 3
- 1.6 Future Directions 4

2 Definitions 5

3 Communication Protocols for the O-MI Standard 6
- 3.1 HTTP and HTTPS Interface 6
- 3.2 RESTful Interface 7
- 3.3 WebSocket 7
- 3.4 WSDL/SOAP Interface 8

4 Messaging Objects 9
- 4.1 Requests 10
  - 4.1.1 Read Request 10
  - 4.1.2 Write Request 12
  - 4.1.3 Call Request 13
  - 4.1.4 Delete Request 14
  - 4.1.5 Cancel Request 15
  - 4.1.6 Response Element 16

5 Typical Uses of O-MI Requests 18
- 5.1 One-Time Read Request 18
- 5.2 Subscription Request 19
- 5.3 Write Request 20
- 5.4 Call Request 20
- 5.5 Delete Request 21
- 5.6 Data Source and Metadata Discovery 22
- 5.7 Publishing Data and Data Sources 22
- 5.8 Synchronous Dialog 23
- 5.9 Specifying a List of Destination O-MI Nodes 23

6 Error Handling 25
- 6.1 Communication-Level Errors 25
- 6.2 O-MI Content-Level Errors 25

A O-MI XSD Schema (Normative) 26

B Example Messages 32

C Example Messages for Method Publication and Call 36

D JSON Mapping and Examples 41

Abbreviations 43

List of Tables

Table 1: O-MI Units and Formats 9
Table 2: Attributes of omiEnvelope 9
Table 3: Attributes of Read Request 10
Table 4: Child Elements of Read Request 11
Table 5: Attributes of Write Request 12
Table 6: Child Elements of Write Request 13
Table 7: Attributes of Call Request 14
Table 8: Child Elements of Call Request 14
Table 9: Attributes of Delete Request 15
Table 10: Child Elements of Delete Request 15
Table 11: Child Elements of Cancel Request 16
Table 12: Attributes of Response Element 16
Table 13: Child Elements of Response Element 17
Table 14: Attribute and Child Element Descriptions for One-Time Read Request 18
Table 15: Attribute and Child Element Descriptions for Subscription Request 19
Table 16: Attribute and Child Element Descriptions for Call Request 21

List of Examples

Example 1: How a Minimal O-MI Message (No Payload) can be Sent using the UNIX "curl" Utility 6
Example 2: Minimal Response to an O-MI Request 7
Example 3: Smallest Possible Response to a Successful Request 16
Example 4: Publishing "Object", "InfoItem", and "Value" using O-DF Semantics 23
Example 5: Read Request Example 32
Example 6: Read Response 32
Example 7: Read Response Metadata Example 33
Example 8: Multiple Payload Response Example 33
Example 9: Write Request 34
Example 10: Error Response 34
Example 11: Cancel Request 35
Example 12: Publish Method by *write* Request 36
Example 13: Read Request and Response for Method 37
Example 14: Read Request and Response for Method MetaData 38
Example 15: Call Request and Response for Method 39

Preface

The Open Group

The Open Group is a global consortium that enables the achievement of business objectives through technology standards. Our diverse membership of more than 700 organizations includes customers, systems and solutions suppliers, tools vendors, integrators, academics, and consultants across multiple industries.

The mission of The Open Group is to drive the creation of Boundaryless Information Flow™ achieved by:

- Working with customers to capture, understand, and address current and emerging requirements, establish policies, and share best practices
- Working with suppliers, consortia, and standards bodies to develop consensus and facilitate interoperability, to evolve and integrate specifications and open source technologies
- Offering a comprehensive set of services to enhance the operational efficiency of consortia
- Developing and operating the industry's premier certification service and encouraging procurement of certified products

Further information on The Open Group is available at www.opengroup.org.

The Open Group publishes a wide range of technical documentation, most of which is focused on development of Standards and Guides, but which also includes white papers, technical studies, certification and testing documentation, and business titles. Full details and a catalog are available at www.opengroup.org/library.

This Document

This document is The Open Group Internet of Things (IoT) Standard for the Open Messaging Interface (O-MI), Version 2.0. It has been developed and approved by The Open Group.

This document specifies Version 2.0 of the O-MI standard. It is based on Version 1.0 that was published in 2017. Particular care has been taken to, as far as possible, keep Version 2.0 backwards-compatible with earlier versions.

This document is structured as follows:

- Chapter 1 provides an introduction to the standard and describes conformance requirements, normative references, and terminology
- Chapter 2 contains Definitions
- Chapter 3 specifies how O-MI messages are communicated using HTTP POST/GET operations, as well as for WSDL/SOAP-based communication

- Chapter 4 provides detailed specifications for the usage, attributes, and elements of the basic O-MI requests, as well as for response messages
- Chapter 5 describes and specifies the most commonly used types of operations performed with O-MI requests
- Chapter 6 describes how errors should be handled
- Appendix A details the O-MI XSD schema
- Appendix B contains example messages
- Appendix C shows example messages for method publication and call
- Appendix D shows possible JSON mappings

Trademarks

ArchiMate®, DirecNet®, Making Standards Work®, Open O® logo, Open O and Check® Certification logo, OpenPegasus®, Platform 3.0®, The Open Group®, TOGAF®, UNIX®, UNIXWARE®, and the Open Brand X® logo are registered trademarks and Agile Architecture Framework™, Boundaryless Information Flow™, Build with Integrity Buy with Confidence™, Dependability Through Assuredness™, Digital Practitioner Body of Knowledge™, DPBoK™, EMMM™, FACE™, the FACE™ logo, IT4IT™, the IT4IT™ logo, O-AAF™, O-DEF™, O-HERA™, O-PAS™, Open FAIR™, Open Platform 3.0™, Open Process Automation™, Open Subsurface Data Universe™, Open Trusted Technology Provider™, O-SDU™, Sensor Integration Simplified™, SOSA™, and the SOSA™ logo are trademarks of The Open Group.

Java® is a registered trademark of Oracle and/or its affiliates.

All other brands, company, and product names are used for identification purposes only and may be trademarks that are the sole property of their respective owners.

Acknowledgements

The Open Group gratefully acknowledges the contribution of the following people in the development of this standard.

Contributing Members of The Open Group IoT Work Group

- Andrea Buda, Aalto University
- Kary Främling*, Aalto University* (IoT Work Group Chair)
- Chris Harding, Lacibus
- Robert Hellbach, BIBA
- Lauri Isojärvi, Aalto University
- Tuomas Kinnunen, Aalto University
- Dimitris Kiritsis, EPFL
- Manik Madhikermi, Aalto University
- Jussi Pirilä, Aalto University
- Edward Roberts, Elparazim
- Ron Schuldt, Data-Harmonizing LLC

* Primary Contributor

The Open Group Staff

- Michelle Supper, Director, The Open Group IoT Work Group

Referenced Documents

(Please note that the links below are good at the time of writing but cannot be guaranteed for the future.)

Normative References

Normative references for this standard are defined in Section 1.4.

Informative References

The following document is referenced in this standard:

- An Introduction to Internet of Things (IoT) and Lifecycle Management, White Paper (W167), May 2016, published by The Open Group; refer to: www.opengroup.org/library/w167

1 Introduction

1.1 Objective

The Open Group Standards for the Internet of Things (IoT) have been developed to fill an interoperability gap identified in the context of the IoT. In order to achieve its full potential, the IoT requires a trusted and secure, open, and unified infrastructure for true interoperability. Without this, continued lack of trust and parallel development of disparate solutions, technologies, and standards will lead it to become an ever-increasing web of organization and domain-specific intranets. The IoT standards provide this infrastructure, as explained in greater detail in the White Paper: An Introduction to Internet of Things and Lifecycle Management.

1.2 Overview

The IoT tends to be defined in different ways by different people. For the IoT Work Group of The Open Group, IoT products and systems can have varying degrees of "intelligence", from barcodes or RFID tags that have only an identifier to smart houses, vehicles, and other products that have advanced sensing and actuating capabilities, as well as powerful processing, memory, and communication capabilities. The lifecycle management aspect of the O-MI standard requires that it must be able to provide interoperability between products and with all other information systems that consume or provide information that is relevant to the product lifecycle. Despite the focus on product lifecycles, it is also the intention that the IoT standards would be applicable to lifecycles of "anything", such as humans, services, projects, electronic documents, etc. Therefore, the O-MI standard has been specified in the most generic way possible.

The O-MI connectivity model is similar to that of the World-Wide Web (WWW). Where the WWW uses the HTTP protocol for transmitting HTML-coded information mainly intended for human users, O-MI requests are used for transmitting lifecycle-related information mainly intended for automated processing by information systems. In the same way as HTTP can be used for transporting payloads also in other formats than HTML, O-MI requests can be used for transporting payloads in almost any format, such as XML, JSON, CSV, etc.

A defining characteristic of the O-MI standard is that O-MI nodes do not have predefined roles, as it follows a "peer-to-peer" communications model. This means that any O-MI node can act both as a "server" and as a "client" and therefore communicate directly with each other or with back-end servers. Typical examples of exchanged data are sensor readings, alarm or lifecycle events, requests for historical data, notifications about availability of new data, changes to existing data, etc.

The main properties and requirements for this standard are:

1. *O-MI messages can be transported using most "lower-level" protocols*. This signifies that protocols such as HTTP, SOAP, SMTP, files on mass storage media such as USB sticks, text messages on mobile phones, etc. can be used for transporting O-MI messages.

2. *Five possible operations*: *read, write, call, delete, and cancel*. Read is for immediate retrieval of information and for placing subscriptions for deferred retrieval of information from an O-MI node. Write is for sending information updates to O-MI nodes. Call is for enabling remote method calls. Delete is for requesting the removal of information from the destination node. Cancel is for cancelling subscriptions before they expire.

3. *O-MI nodes can request current and historical data with an immediate response*. This is done by the read operation. Information is retrieved as a response message.

4. *O-MI nodes can send data to each other at any time*. This is done by the write operation.

5. *Subscriptions can be made for deferred retrieval of data from other O-MI nodes*. This is done by the read operation if the *interval* parameter has been set. If a callback address is provided, then the data is sent using an O-MI response message at the requested interval. If no callback address is provided, then the data can be retrieved (polled) by issuing a new read request with the ID of the subscription (this is particularly useful if the requesting node is behind a firewall or using NAT).

6. *Allowing different payload formats both for requests and responses.* An O-MI message can transport actual information using any text-based format (standardized, proprietary, etc.) that can be embedded into an XML message. It is even possible to use different payload formats in different *return* elements of an O-MI response.

7. *All requests and responses specify a Time-to-Live (TTL).* If the message has not been delivered to the "next" O-MI node before the TTL expires, then the message should be removed and an error message returned or sent to the message originator, if possible.

8. *Enable synchronous communication between nodes*. Any response message can include a new request, which is useful, for example, in control applications. It also provides a possibility to perform "client-initiated" communication with nodes that are located behind firewalls or NATs.

9. *Publication and discovery of data sources, services, and metadata.* Publication of new data sources, services, and metadata can be done with the write operation. "RESTful" URL-based (HTTP GET) queries (in addition to read operations) allow the discovery of them, including discovery by search engines.

   Note: Format and semantics used by different data sources, services, metadata, etc. are not part of the O-MI standard; their format and semantics are specified by other standards, such as the Open Data Format (O-DF) standard or similar.

10. *All requests can specify a list of target O-MI nodes*. The receiving node(s) are then responsible for re-routing the request to the target O-MI nodes or sending back an error message to the requesting O-MI node in case of failure.

1.3 Conformance

This standard specifies conformance requirements for O-MI messages, O-MI nodes, and O-MI messaging interfaces in O-MI nodes. Chapters 2 through 6 and Appendix A are normative. Appendices B through D are informative.

1.4 Normative References

The following standards contain provisions which, through references in this standard, constitute provisions of the O-MI standard. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this standard are encouraged to investigate the possibility of applying the most recent editions of the standards listed below. Members of IEC and ISO maintain registers of currently valid International Standards.

- IETF RFC 2616: Hypertext Transfer Protocol – HTTP/1.1, June 1999; refer to: www.ietf.org/rfc/rfc2616.txt
- IETF RFC 6455: The WebSocket Protocol, December 2011; refer to: https://tools.ietf.org/html/rfc6455
- IETF RFC 7235: Hypertext Transfer Protocol (HTTP/1.1): Authentication, June 2014; refer to: https://tools.ietf.org/html/rfc7235
- Open Data Element Framework (O-DEF™), Version 1.0, The Open Group Standard (C163), May 2016, published by The Open Group; refer to: www.opengroup.org/library/c163
- Open Data Format (O-DF), The Open Group Internet of Things (IoT) Standard (C14A), October 2014 (with Technical Corrigenda updates in May 2016 and September 2017), published by The Open Group; refer to: www.opengroup.org/library/c14a
- Open Data Format (O-DF), Version 2.0, The Open Group Internet of Things (IoT) Standard (C19D), December 2019, published by The Open Group; refer to: www.opengroup.org/library/c19d
- Open Messaging Interface (O-MI), The Open Group Internet of Things (IoT) Standard (C14B), October 2014 (with updates in May 2016 and September 2017), published by The Open Group; refer to: www.opengroup.org/library/c14b
- XML Schema Part 2: Datatypes Second Edition, Ed. Biron & Malhotra; refer to: www.w3.org/TR/xmlschema-2

1.5 Terminology

For the purposes of this standard, the following terminology definitions apply. When used in this way, these terms will always be shown in ALL CAPS; when these words appear in ordinary typeface, they are intended to have their ordinary English meaning.

Can — Describes a permissible optional feature or behavior available to the user or application. The feature or behavior is mandatory for an implementation that conforms to this document. An application can rely on the existence of the feature or behavior.

May — Describes a feature or behavior that is optional for an implementation that conforms to this document. An application should not rely on the existence of the feature or behavior. An application that relies on such a feature or behavior cannot be assured to be portable across conforming implementations. To avoid ambiguity, the opposite of “may” is expressed as “need not”, instead of “may not”.

Must — Describes a feature or behavior that is mandatory for an application or user. An implementation that conforms to this document shall support this feature or behavior.

Shall — Describes a feature or behavior that is mandatory for an implementation that conforms to this document. An application can rely on the existence of the feature or behavior.

Should — For an implementation that conforms to this document, describes a feature or behavior that is recommended but not mandatory. An application should not rely on the existence of the feature or behavior. An application that relies on such a feature or behavior cannot be assured to be portable across conforming implementations. For an application, describes a feature or behavior that is recommended programming practice for optimum portability.

Will — Same meaning as “shall”; “shall” is the preferred term.

1.6 Future Directions

This standard will continue to be maintained by the IoT Work Group of The Open Group Platform 3.0™ Forum. It may be revised to correct errors or to incorporate changes based on implementation experience.

Also, it contains normative provisions for use of XML, and an informative example indicating how JSON could be used. A future standard could contain normative provisions for use of JSON.

2 Definitions

Merriam-Webster's Collegiate Dictionary should be referenced for terms used in this standard.

3 Communication Protocols for the O-MI Standard

In addition to basic read and write operations for querying and writing information (in similar ways to GET/POST in HTTP or get/set-methods in Java® programming), the subscription concept is a cornerstone of the O-MI standard. The conceptual framework used here is the Observer design pattern, which is familiar to most programmers, and notably Java programmers.

Using the Observer pattern, an O-MI node can add itself as an observer of events that occur at another O-MI node. In this sense the O-MI standard differs from, for example, the Java Message Service (JMS) or the Message Queuing Telemetry Transport (MQTT) standard that are based on the Publish-Subscribe model. For many applications the Observer and the Publish/Subscribe models can be used in quite similar ways.

However, the Publish-Subscribe model usually assumes the usage of a "high-availability server", which the Observer pattern doesn't. This is why the Observer model is more suitable for many IoT applications where products and different information systems might communicate with each other directly. Furthermore, the Observer pattern can also be used for implementing a Publish-Subscribe structure.

O-MI messages can be communicated through any communication protocol, removable storage media, or similar that can handle XML documents or XML strings. The default and recommended communication protocol is HTTP or HTTPS with the POST request. O-MI nodes SHOULD implement this communication method. In many practical implementations in recent years, the WebSocket protocol has also been used. Therefore, a new section on the use of WebSocket with O-MI messages has been added (see Section 3.3).

O-MI messages can also be communicated using WSDL/SOAP[1] protocols according to this standard. O-MI nodes NEED NOT implement WSDL/SOAP as the protocol for O-MI messages.

3.1 HTTP and HTTPS Interface

O-MI messages MAY be communicated using HTTP and HTTPS protocols. If so, O-MI nodes SHOULD send O-MI messages as the payload of an HTTP POST request. If a parameter is used in the HTTP POST request for the O-MI payload, then the name of that parameter SHALL be called "*msg*". Example 1 shows how a minimal O-MI message can be sent with HTTP POST using the UNIX® "curl" utility, where the URL of the O-MI node is https://smartcampus.org.aalto.fi/omi/.[2] If it is received correctly, then a reply similar to that in Example 2 should be received.

Example 1: How a Minimal O-MI Message (No Payload) can be Sent using the UNIX "curl" Utility

```
curl http://dialog.hut.fi/omi/ --data msg="<?xml version="1.0" encoding="UTF-
8"?><omiEnvelope version="2.0" ttl="10"><read><msg></msg></read></omiEnvelope>"
```

[1] These are often called "Web Services" but the Web Service concept is not uniquely defined and tends to be used for signifying different things by different people and organizations.

[2] The URL "https://smartcampus.org.aalto.fi/omi/" is provided as an example of a valid URL of an O-MI node. The reader should not assume that a valid O-MI node would be available at that address, nor that it would return the content shown in this standard.

Example 2: Minimal Response to an O-MI Request

```
<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope version="2.0" ttl="0">
    <response>
        <result>
            <return returnCode="200"></return>
        </result>
    </response>
</omiEnvelope>
```

The HTTP GET operation with a "*msg*" parameter containing an O-MI request MAY also be implemented by O-MI nodes. However, this is not the recommended way of making O-MI requests due to the size limitations of HTTP GET requests that may vary depending on the HTTP server used.

3.2 RESTful Interface

The O-MI interface has been designed to be RESTful whenever possible. This signifies that O-MI messages are intended to be as self-contained as possible, thereby making it possible for O-MI nodes to keep and maintain as little information about session and communication states as possible.

For the publication and discovery of data sources, O-MI nodes SHOULD also implement a URL-based (HTTP GET) mechanism for retrieving a list of available information in a hierarchical way, as described in Chapter 4 of the O-DF standard, Version 2.0.

Contrary to most RESTful specifications, the O-MI interface does not use PUT and DELETE methods because that would create an explicit link between the O-MI interface and the HTTP protocol, which would go against the functional requirements set out for the O-MI standard.

3.3 WebSocket

The WebSocket protocol (IETF RFC 6455) MAY be used as the underlying protocol for O-MI messages. Use of WebSocket SHALL be indicated by a URL that has "*ws:*" as its scheme. Correspondingly, the "*wss:*" scheme indicates that a TLS-secured WebSocket connection SHALL be used. For instance, the URL "*wss://dialog.hut.fi/omi/node/*" addresses an O-MI node that uses secure WebSocket as the underlying protocol.

In practice, this signifies that the first connection needs to be made using HTTP/HTTPS and then an HTTP Upgrade request issued for establishing a WebSocket connection as explained in IETF RFC 6455.

WebSocket support was introduced mainly for enabling "immediate callback" functionality provided by setting *callback*="0", which signifies that results of subscriptions MUST be sent over the open WebSocket connection.

An O-MI node MAY accept any number of requests via the same connection. Multiple connections CAN be used to distinguish results of different requests from each other.

"*ws:*" and "*wss:*" schemes MAY also be used in callback URLs.

3.4 WSDL/SOAP Interface

If an O-MI node uses the WSDL/SOAP protocols, then the WSDL interface SHALL define one function called “notify”. An O-MI node SHALL use the “notify” function whenever it intends to communicate O-MI messages using SOAP communication.

O-MI nodes SHALL use the “notify” function both for requests and responses in the same way as the HTTP POST protocol described in Section 3.1. If WSDL/SOAP is used to communicate O-MI messages, the callback URL SHALL point to a WSDL service with a function named “notify”.

The notify function description is as follows:

```
String notify(String msg)
```

The parameter *msg* SHALL be the O-MI request or response as an XML string complying with the “O-MI XML Schema”. The responding O-MI node SHALL return an XML structure complying with the “O-MI XML Schema”.

4 Messaging Objects

The XML schema “omi.xsd” in Appendix A provides a formal specification for an O-MI message. If there is any conflicting information between the chapters of this standard and the O-MI schema file, then the information in the schema file is to be used. The schema file is included in Appendix A.

The following units or formats SHALL be used in the O-MI message to express the listed quantities. The *xs* namespace is defined in XML Schema Part 2: Datatypes Second Edition.

Table 1: O-MI Units and Formats

Value	Unit/Format
Date	Use xs:Date
Time	Use xs:dateTime
Duration	Use xs:duration It is defined in the O-MI schema where xs:duration should be used. For other durations, such as TTL, xs:double may be used and then the unit is seconds.
Other Values	SI units used by default.

Table 2 defines generic information included in all O-MI requests and responses. They are defined in the *omiEnvelope* element, which is always the highest-level element of an O-MI message. The *omiEnvelope* CAN have only one of the following child elements: *read*, *write*, *call*, *delete*, *cancel*, *response*.

Table 2: Attributes of omiEnvelope

Attribute	Description
ttl	TTL SHALL be present. The TTL (in seconds) for the O-MI requests. The time is counted from the time the O-MI request was received by the O-MI node. The value “0” signifies that a response must be provided while the connection is active (in the case of HTTP, for instance). The value “–1” signifies “forever”. When the TTL expires, O-MI nodes SHOULD answer requests with an error response.
version	Each O-MI message SHALL have a version number defined. The version number defines the O-MI version used.
authorization	An authorization attribute MAY be present. The value of this attribute SHALL be formatted as specified for the “Authorization” header field in IETF RFC 7235, §4.2 (Authorization). The intended use of this attribute is for forwarding authorization information to the application level, even though the actual authorization may have been performed on the HTTP server level, for instance.
anyAttribute	Other attributes MAY be included but their use is not specified by this standard.

4.1 Requests

O-MI requests SHALL be read, write, call, delete, or cancel requests. O-MI requests MAY define the recipient O-MI nodes for the request. If so, the receiving O-MI node SHOULD send the request to these nodes for execution. If no nodes are defined, the recipient of the request is the only intended recipient. The receiving O-MI node SHOULD include information about which node(s) provided which of the results in the response.

O-MI requests CAN be one-time, or for repeated data updates (subscriptions). *One-time requests* are only answered once (see Section 5.1) while *subscription requests* are answered several times using a callback service provided by the requesting O-MI node (see Section 5.2).

4.1.1 Read Request

Read requests are used to request data or to set up subscription requests for data.

An O-MI node SHALL answer each read request it receives with the requested data or with an error message.

The responding O-MI node SHOULD take care of data persistency of requests. It SHOULD buffer a subscription request with a callback response mechanism until the request is completely fulfilled.

The responding O-MI node SHALL NOT assume that the requesting O-MI node keeps track of placed requests.

The responding O-MI node SHALL answer the request with an O-MI response or with a communication error code in case the request failed on communication connection level.

The response message SHALL NOT contain any other data than requested data, or error information about the processing of the request.

Table 3: Attributes of Read Request

Attribute	Description
callback	Defines a URL to an O-MI node with a callback service where O-MI responses are received. SHALL NOT be present for immediate requests. MAY be present for subscription requests. For subscription requests, if no callback address is provided, then the requested data SHOULD be stored for subsequent retrieval using a read message with the corresponding *requestID*. The special value “0” (zero) has been reserved for specifying that responses SHALL be sent over the same open communication channel through which the request was received. Such a channel could, for instance, be an open WebSocket or TCP connection.
msgformat	SHOULD be present. Text string indicating the format of the payload in “*msg*”; e.g., “odf”, “csv”, “obix”, or similar. For the moment these are free-format text strings, which might become a challenge for interoperability. However, at the time of writing this standard, it is not clear what other convention would be more suitable for this purpose.

Attribute	Description
targetType	MAY be present. Currently defined types are “node” or “device”. In applications dealing with physical devices, using “device” indicates that if the message “target object” is some kind of device connected to an O-MI node, then try to get the requested value directly from the device.
interval	The subscription interval. SHALL be present for subscription requests. SHALL NOT be present for all other than subscription requests. O-MI nodes SHOULD answer positive value subscriptions with the value used as the interval between responses (in seconds). The responding O-MI node SHOULD answer the request as close to the interval as possible. Answering the request exactly on the interval may not be possible. The responding O-MI node SHOULD answer the request at every interval, even if data is not available. If data is not available, the responding O-MI node SHOULD answer the request with an error for those intervals. O-MI nodes SHOULD answer value 0 subscriptions as often as data can be read from the target. Typically used for specific diagnostics and performance monitoring applications. (May cause unnecessary data traffic.) O-MI nodes SHALL answer value –1 subscriptions every time the subscribed target publishes/pushes a new value. O-MI nodes SHALL answer value –2 (minus 2) subscriptions every time the subscribed target (typically a new information source or device) connects to an O-MI node.
oldest	Retrieve the given number of the oldest available historical data. MAY be present.
begin	Retrieve data from the date indicated by the value of this attribute. MAY be present. If no *end* value is provided, then retrieve all available data after *begin*.
end	Retrieve data until the date indicated by the value of this attribute. MAY be present. If no *begin* value is provided, then retrieve all data available until *end*.
newest	Retrieve the newest available number of historical data available. MAY be present.
all	Retrieve all available values. When this attribute is set to “true”, there SHALL NOT be any oldest, begin, end, and newest attributes in the same request. If a request is received with any of those attributes, then they SHALL be ignored. MAY be present.
maxlevels	The maximum number of levels to return from a hierarchical data model, such as the one used by the O-DF standard. MAY be present.

If oldest/newest is used together with begin/end, then the result set SHALL first be restricted by begin/end and then oldest/newest applied to the resulting subset.

Table 4: Child Elements of Read Request

Child Element	Description
nodeList	List of recipient O-MI nodes. MAY be present if the request is directed to predefined O-MI nodes other than the O-MI node that received the request.

Child Element	Description
requestID	List of request IDs of earlier subscriptions. MAY be present if this read request is sent for retrieving data resulting from an existing subscription(s) that was issued without a callback address.
msg	Actual payload of the request, specifying what data is being requested.

4.1.2 Write Request

Write requests are used to write various data to different systems, such as sensor values and information extracted from databases or other systems.

An O-MI node SHALL answer each write request that it receives with a confirmation that the write was successful or with an error message either immediately or to the provided callback address.

The responding O-MI node SHOULD take care of data persistency of requests even in the case of network interruptions, node crashes, or similar situations.

The responding O-MI node SHOULD buffer a request with callback response mechanisms until the request could be completely fulfilled.

An O-MI node SHOULD NOT respond to a write request as successful before the write can be guaranteed. (It is not always possible to guarantee the write was successful, and in such a case the O-MI node SHOULD respond successfully when the request is communicated for writing successfully.) The responding O-MI node SHALL NOT assume that the requesting system keeps track of placed requests.

The responding O-MI node SHALL answer the request with an O-MI message or with an HTTP error code in case the request failed on communication connection level.

If there are any subscriptions to information affected by the write request, then the responding O-MI node SHALL generate corresponding event(s) in order to reflect the value change.

Table 5: Attributes of Write Request

Attribute	Description
ttl	TTL SHALL be present. The responding O-MI node SHOULD write the request to the targets before the TTL expires. If the TTL expires, the responding O-MI node SHALL answer it with an error response. The response MAY contain both error messages and confirmation messages if the write was successful for some targets and failed for others.
callback	SHALL be present for callback requests. SHALL NOT be present for immediate requests. SHALL define a URL to a callback service where O-MI responses are received. The special value “0” (zero) SHALL be used in the same way as for *read* request.
msgformat	Same as for read request.

Attribute	Description
targetType	MAY be present. Currently defined types are “node” or “device”. In applications dealing with physical devices, using “device” indicates that if the message “target object” is some kind of device connected to a node, then try to write the requested value directly to the “device” as soon as possible.

Table 6: Child Elements of Write Request

Child Element	Description
nodeList	Same as for read request.
requestID	List of request IDs of earlier request(s). MAY be present. However, the current specification does not define how that list of request IDs is to be used in write requests. Therefore, any use of this feature will be application-specific with this current specification. Future specifications may include recommendations for this feature.
msg	Actual payload of the request, specifying what data is being written and corresponding values.

4.1.3 Call Request

Call requests correspond to *method* calls. A method is an O-DF *InfoItem* (information models other the O-DF model might be supported in the future). The *msg* child element SHALL provide the O-DF tree that specifies the method that is called, as well as the expected input parameters of the method.

A *method InfoItem* declared in the O-DF standard SHALL have the following properties:

- A *name* that specifies the name of the *method*
- A static value *odf:Method* (`<value>odf:Method</value>`)
- A *MetaData* block that SHALL contain at least:
  - An *InfoItem* called *odf:InfoItemType* with the static value *odf:Method* (`<InfoItem name="odf:InfoItemType"><value>"odf:Method"</value></InfoItem>`)

    This requirement partially overlaps with the `<value>` of the *method InfoItem*. The reason for this overlap is that *InfoItem* value and *InfoItem MetaData* are typically retrieved with two separate *read* requests, so including this information also in the *MetaData* means that all necessary information about the method can be retrieved by a single *MetaData read* request.
  - An *InfoItem* called *odf:Parameter* (`<InfoItem name="odf:Parameter">…`)

    The value of this *InfoItem* SHALL be an O-DF structure with `<Objects>` as its uppermost element, which specifies the expected structure of the possible input parameters to the method.

— An *InfoItem* called *odf:ReturnType* (`<InfoItem name="odf:ReturnType">…`)

The value of this *InfoItem* SHALL be an O-DF structure with `<Objects>` as its uppermost element, which specifies the expected structure returned by the method.

A *method InfoItem* is typically published to O-MI nodes using a *write* request that contains this information. Different implementations of O-MI nodes MAY also use other means of publishing O-MI methods.

An O-MI node SHALL answer each *call* request that it receives with a corresponding result or with an error message either immediately or to the provided callback address.

The responding O-MI node SHALL answer the request with an O-MI message or with an HTTP error code in case the request failed on communication connection level.

If there are any subscriptions to information affected by the *call* request, then the responding O-MI node SHALL generate corresponding event(s) in order to reflect the value change.

Table 7: Attributes of Call Request

Attribute	Description
callback	SHALL be present for callback requests. SHALL NOT be present for immediate requests. SHALL define a URL to a callback service where O-MI responses are received. The special value “0” (zero) SHALL be used in same way as for *read* request.
msgformat	Same as for *read* request.
targetType	SHALL NOT be present. For the moment, call requests are assumed to be handled by the target O-MI node.

Table 8: Child Elements of Call Request

Child Element	Description
nodeList	SHALL NOT be present.
requestID	SHALL NOT be present.
msg	SHALL be present. Actual payload of the request, specifying the input parameters of the method call.

4.1.4 Delete Request

Delete requests MAY be used to delete data or entire information structures from O-MI nodes.

An O-MI node SHALL answer each *delete* request that it receives with a confirmation that the delete was successful or with an error message either immediately or to the provided callback address.

An O-MI node SHOULD NOT respond to a *delete* request as successful before the delete can be guaranteed. (It is not always possible to immediately guarantee that the delete was successful,

and in such a case the O-MI node SHOULD respond successfully when the request has been completed successfully.)

The responding O-MI node SHALL answer a *delete* request with an O-MI message or with an HTTP error code in case the request failed on communication connection level.

If there are any subscriptions to information affected by the *delete* request, then subsequent messages related to those subscriptions SHALL have empty value(s) (`<value></value>` or `<value/>`) for the deleted *InfoItems*.

Table 9: Attributes of Delete Request

Attribute	Description
callback	SHALL be present for callback requests. SHALL NOT be present for immediate requests. SHALL define a URL to a callback service where O-MI responses are received. The special value “0” (zero) SHALL be used in same way as for *read* request.
msgformat	Same as for *read* request.
targetType	SHALL NOT be present. For the moment, delete requests are assumed to be handled by the target O-MI node.

Table 10: Child Elements of Delete Request

Child Element	Description
nodeList	SHALL NOT be present.
requestID	SHALL NOT be present.
msg	SHALL be present. Actual payload of the request, specifying what information is to be deleted.

4.1.5 Cancel Request

An O-MI node SHALL answer each cancel request that it receives with a confirmation that the cancellation of a request was successful or with an error message.

An O-MI node receiving a cancel request for an original request with a recipient list SHALL direct the cancel request to the recipients.

A cancel request SHALL define the ID of the request(s) to cancel.

An O-MI node receiving a cancel request SHALL answer it with an O-MI message or with an HTTP error code in case the request failed on communication connection level.

Table 11: Child Elements of Cancel Request

Child Element	Description
requestID	SHALL be present. List of request IDs of the request(s) to cancel.
nodeList	Same as for read request. MAY be present if the request(s) is cancelled at predefined O-MI nodes.

4.1.6 Response Element

Response content varies with the request type and usage of the response. This section defines the generic response structures.

A response SHALL contain at least one result. Every result object SHALL contain a return object that indicates whether the request was successful or not using HTTP status codes; i.e., “200” if successful. If there are several result objects in a response, then they typically correspond to different callback requests.

Example 3: Smallest Possible Response to a Successful Request

```
<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope version="2.0" ttl="0">
    <response>
        <result>
            <return returnCode="200"></return>
        </result>
    </response>
</omiEnvelope>
```

An O-MI node that receives a callback request SHALL send a response immediately as a return value to the request and the response SHALL contain a request ID. If the request is received correctly, this response message SHOULD NOT contain anything else.

An O-MI node that receives a callback or immediate request that is unsuccessful or that could not be received properly SHALL send to the requester an immediate response that contains error information.

All result data in a response object SHALL be in the *msg* element. The *msgformat* attribute SHOULD contain a text string that identifies the payload format used, such as “odf”, “csv”, “obix”, or similar. A responding O-MI node SHALL NOT include data that was not requested in a response.

If a request encountered problems during its execution or some data was not available, the responding O-MI node SHALL send a corresponding result that contains information about the problem.

Response objects MAY contain both successful and unsuccessful result objects.

Table 12: Attributes of Response Element

Attribute (result)	Description
msgformat	Same as for read request.

Attribute (result)	Description
targetType	MAY be present. Currently defined types are “node” or “device”. In applications dealing with physical devices, the value of this attribute indicates whether requested value(s) come from the device directly or whether it is the last data known by the node. If omitted, the default value is “node”.
format (*requestID*)	MAY contain text string that specifies the format of the request ID used; e.g., “odef, …”.

Table 13: Child Elements of Response Element

Child Element	Description
return	SHALL be present. Return code of the request. MAY contain a “description” attribute, which is then a text string describing the error.
requestID	SHALL be present if this message is the result of a callback request, else SHALL NOT be present.
msg	MAY be present. Actual payload of the result.
nodeList	MAY be present. List of O-MI nodes that have provided the result.
omiEnvelope	MAY be present. O-MI envelope containing a new request from the responding O-MI node to the originator O-MI node. The purpose of this child element is to enable a synchronous, direct dialog between two O-MI nodes using an already open connection. It is also useful for communicating with O-MI nodes that are behind firewalls and that therefore must initiate the communication.

5 Typical Uses of O-MI Requests

The typical uses of O-MI requests given in this chapter should cover most of the actual needs and scenarios. However, the O-MI standard has been specified with the intention to be as generic and flexible as possible for most potential information architectures that might occur or be needed for an IoT as well as for lifecycle management. Therefore, these typical uses should not be considered as the only possible ones even though they do represent most of the actual implementations of the O-MI standard so far.

5.1 One-Time Read Request

An O-MI node SHALL answer a one-time read request once only. The answer can be provided using immediate responses or a callback service. If a callback is used, the requesting O-MI node SHALL only call the callback point once.

An O-MI node SHALL answer a one-time read request with a response that contains the requested data if the data was available and the requester had appropriate access rights to that data, and with a response that contains an error message if the data was not available.

If a one-time read request is partially successful and partially unsuccessful, then the *response* element SHALL contain separate *return* elements with corresponding *returnCode* attribute values. Each return element SHALL contain the corresponding data structure. If the O-DF standard is used, then the `<value>` elements SHALL be empty for unsuccessful parts of the request.

Immediate response requests can typically be used for GUIs and components where there is an active user, to be able to respond to the user in an interactive way. Immediate response requests are answered immediately to the requesting system without using a callback.

Immediate response requests need not be buffered, and the requesting O-MI node SHOULD be waiting idle until the request is either fulfilled or results in an error. If the defined TTL expires before the request can be processed or before it causes an error response, the responding O-MI node SHALL answer the request with an error response.

Table 14: Attribute and Child Element Descriptions for One-Time Read Request

Attribute/ Child Element	Description
interval	SHALL NOT be present.
ttl	When the defined TTL expires, the responding O-MI node SHALL answer the request with an error response and the data retrieved so far.
callback	SHALL NOT be present in the immediate response request.

5.2 Subscription Request

The O-MI standard allows one-off or standing information request subscriptions to be made in a loosely-coupled, lightweight, and generic way. Subscriptions can be made for receiving updates at regular intervals or on an event basis – when the value or status changes for the information subscribed to.

Subscription requests SHOULD use a callback point as the request may be answered several times. If no callback address is provided, then the responding O-MI node SHOULD store the data acquired for the subscription for later retrieval with a read message that contains the corresponding *requestID*.

An O-MI node receiving a subscription request SHALL answer it by an immediate response message containing a *requestID*. The response SHALL NOT return any data.

A callback service is typically used to answer requests that cannot be fulfilled immediately and requests where the requesting system waits in an idle state until the request is fulfilled.

An O-MI node receiving a subscription request with a callback O-MI node SHALL answer it when the request is fulfilled by calling the callback service, either if the request could be fulfilled successfully or if it resulted in an error. An O-MI node receiving a subscription request MAY send responses and actual values in any order according to data availability.

As for one-time read requests, if a subscription request is partially successful and partially unsuccessful, then the *response* element SHALL contain separate *return* elements with corresponding *returnCode* attribute values. Each *return* element SHALL contain the corresponding data structure. If the O-DF standard is used, then the `<value>` elements SHALL be empty for unsuccessful parts of the request.

An O-MI node receiving a subscription request (or any system connected to it and set up for the purpose) SHOULD generate the *requestID*. An O-MI node sending a subscription request SHOULD NOT include a request ID in it. An O-MI node receiving a subscription request SHALL send an immediate response that MAY contain error information but SHALL NOT contain actual request data. If it contains error information the response SHOULD NOT contain a request ID. The request is considered failed.

An O-MI node receiving a subscription request with a callback O-MI node MAY call the callback O-MI node several times for the same request. It SHOULD call the callback point every time data is available to answer the request, or when an error occurred. It MAY call the callback point with incomplete response messages where only part of the requested data is included.

Table 15: Attribute and Child Element Descriptions for Subscription Request

Attribute/ Child Element	**Description**
interval	SHALL be present.
ttl	TTL defines for how long the subscription is valid. The receiving O-MI node SHALL NOT send actual data to the callback after the TTL expires.

Attribute/ Child Element	Description
callback	SHALL be defined for callback responded requests. SHALL define the URL to the callback O-MI node to be called by the system when the request is answered.
requestID	SHALL be provided by the responding O-MI node to the originating O-MI node in the immediate response if the subscription request is received correctly. If an error occurred, a request ID SHOULD NOT be provided. SHALL be present in all responses to the callback, if there is not an error.

5.3 Write Request

Whereas subscription requests are used for setting up time-limited, loosely-coupled information flows for different pieces of information between O-MI nodes, write requests are rather used for "live forever" requests, such as a machine sending a predefined set of information at predefined intervals and to a predefined callback address of its manufacturer company.

Another example when a write request is needed is if a sporadic event occurs at an O-MI node that needs to be communicated to another O-MI node that hasn't set up a subscription to receiving such events. It is then up to the receiving O-MI node to accept the incoming write request or not.

While read requests correspond to a "client pull" mechanism, write requests correspond to a "client push" mechanism. Subscription requests represent a mixture between these two mechanisms. Which mechanism and request type is the most appropriate to use tends to depend on the application.

It is worth noting that the callback mechanism in write requests is intended to be used for asynchronous notifications about the success or failure of the write request. For instance, if a write request is issued for a mobile device, then the actual write operation may occur a long time after issuing the write request.

Write requests can also be used for publishing data sources, as explained in Section 5.6.

5.4 Call Request

Call requests are sent to a specific method on the target O-MI node. Method publication and calling is done through the following steps:

1. **Publish** the *method* on O-MI node through *write* request (or some implementation-specific mechanism).
2. *Method* needs to be **associated with corresponding service** by the O-MI node implementation.
3. **Discover all** available methods by *read* request on the *Object* that provides the methods. *Method InfoItems* will have the static value `odf:Method`.

4. **Retrieve** the input parameters and result structure of the *method* by issuing *read* request on the *MetaData* of the method.

5. **Call** the *method* using a *call* request.

Example messages are shown for these steps in Appendix C.

An O-MI node SHALL answer a *call* request once only. The answer can be provided using immediate responses or a callback service. If a callback is used, the requesting O-MI node SHALL only call the callback point once.

An O-MI node SHALL answer a *call* request with a response element that contains the result of the call, and with a response that contains an error message if the O-MI node could not produce the result or execute the method correctly.

Table 16: Attribute and Child Element Descriptions for Call Request

Attribute/ Child Element	Description
interval	MAY be present. If present, then *callback* MUST be specified also. Positive *interval* values indicate that the *call* SHOULD be executed by the responding O-MI node as close to the interval as possible. Executing the *call* request exactly on the interval may not be possible. The responding O-MI node SHOULD execute the *call* request at every interval and send the resulting *response* to the *callback* address. With zero (0) *interval*, the responding O-MI node SHOULD execute the call as often as it can or as often as it makes sense (from responding O-MI node's point of view). Typically used for specific diagnostics and performance monitoring applications. (May cause unnecessary data traffic.) Zero interval requests are potential points of attack so particular attention SHOULD be paid to restrict their execution. O-MI nodes SHALL answer *interval* –1 call requests every time the responding O-MI node considers it relevant due to a change in the *call* result. A –2 (minus 2) *interval* SHALL NOT be used with a *call* request.
ttl	When the defined TTL expires, the responding O-MI node SHALL answer the request with an error response and a possibly unfinished *call* result.
callback	MAY be present. MUST be present if *interval* has been specified.

5.5 Delete Request

A *delete* request SHALL result in the removal of the requested information at the receiving O-MI node. The information to delete SHALL be specified in the *msg* sub-element. How the content of the *msg* sub-element is to be interpreted depends on the *msgformat* attribute of *msg*. If the *MsgType* is "odf" (Open Data Format), then information SHALL be deleted from the leaves of the O-DF structure downwards, including the leaves themselves.

An O-MI node SHALL answer a *delete* request once only. The answer can be provided using immediate responses or a callback service. If a callback is used, the requesting O-MI node SHALL only call the callback point once.

An O-MI node MAY answer a *delete* request with a response that contains both error messages and data if parts of the request were successful. However, since the structure for such responses is currently not specified, it is recommended to target *delete* request targets sufficiently precisely so that success or failure of the request provides sufficient information.

5.6 Data Source and Metadata Discovery

The RESTful, URL-based (HTTP GET) mechanism described in Section 3.2 is intended to be the main mechanism of discovering available data sources and metadata about them. An alternative is to issue a read request (typically using HTTP POST).

The way in which different O-MI nodes publish their available data sources and metadata depends on the actual implementation. For instance, in the case of a Smart Home, the O-MI node would typically publish the list of available devices or other information sources about the house, as well as what information is available about those devices and information sources. Finally, metadata information such as units, accuracy, etc. can also be published in the same way.

Another example of information that can be published would be available folders and files for a file repository, together with metadata about the files. However, the semantics and structure of such information are out of the scope of the O-MI standard. Such semantics would be defined by other standards or conventions, such as the O-DF standard, oBIX, or similar. However, in order to simplify the maintenance of an extensible set of application-specific semantic object models, it is recommended that they use URIs for object IDs and for *InfoItem* names.

O-MI nodes SHOULD NOT use read requests (with HTTP POST or HTTP GET with the "*msg*" parameter) for this kind of query because read requests should reply with the entire XML structure that corresponds to the read request. Therefore, issuing a read request to the uppermost level of an O-MI node would conceptually result in a response that contains the entire information hierarchy for all data sources, metadata, and current data values. In practice, O-MI nodes would restrict the amount of information returned based on security settings and other configuration options. Also, since the O-MI standard, Version 2.0, the *maxlevels* attribute of *read* requests allows also the requesting O-MI node to restrict the amount of data retrieved.

5.7 Publishing Data and Data Sources

As mentioned in the previous section, different O-MI nodes may use different mechanisms for publishing their data sources. In addition to those, O-MI nodes MAY allow the usage of write requests for publishing data sources and data.

Example 4 shows an example of an ordinary message using O-DF semantics for publishing the value of information item "FridgeTemperatureSetpoint" of the refrigerator "SmartFridge22334411". The crucial question here is how the receiving O-MI node behaves if either "SmartFridge22334411" or "FridgeTemperatureSetpoint" (or both) are not previously known to the node. Different possibilities are, for instance:

1. Reject the write request, return an error response.
2. Accept the write request, accept and "publish" the new data source(s) "SmartFridge22334411" and/or "FridgeTemperatureSetpoint". If "SmartFridge22334411"

existed already, then just add the new information item “FridgeTemperatureSetpoint” with the given value.

3. Accept or reject the write request depending on whether it comes from “localhost”, a trusted server, or an unknown location. This decision may also depend on whether it is a new Object that is being created or just a new *InfoItem* for an existing Object.

As shown by these three possibilities, the O-MI standard provides the possibility to publish new data sources and data using write requests. However, how this is implemented by the receiving O-MI node and what security policies apply are not in the scope of the O-MI standard.

Example 4: Publishing “Object”, “InfoItem”, and “Value” using O-DF Semantics

```
<?xml version="1.0" encoding="UTF-8"?>
<omi:omiEnvelope ttl="0" version="2.0">
    <omi:write msgformat="odf">
        <omi:msg>
            <Objects>
                <Object type=”…”>
                    <id>SmartFridge22334411</id>
                    <InfoItem name=”FridgeTemperatureSetpoint”>
                        <value>3.5</value>
                    </InfoItem>
                </Object>
            </Objects>
        </omi:msg>
    </omi:write>
</omi:omiEnvelope>
```

5.8 Synchronous Dialog

A *result* element of an O-MI response MAY include a child element called *omiEnvelope* that is, indeed, of type *omiEnvelope*. A receiving O-MI node SHALL use this element if it wants to issue its own request at the same time as it sends its response to the original request. This functionality is provided and needed at least in the following cases:

1. The sender O-MI node of the original request is behind a firewall, it is mobile, or, for some other reason, is never or only occasionally accessible for sending requests to it. This signifies that the original sender O-MI node always SHALL initiate all communication so the only way of sending requests to it is to do it together with a response.

2. For some reason the dialog between the two O-MI nodes needs to be as fast as possible. This could be the case, for instance, when a user is directly interacting with a machine over the Internet. Using this synchronous dialog signifies that the speed of interaction will not be limited by callback overhead or similar.

3. Others: it is expected that future applications will develop new ways of using this functionality.

5.9 Specifying a List of Destination O-MI Nodes

The optional child element *nodeList* can be used in requests for explicitly specifying URIs of O-MI nodes that the receiving O-MI node SHALL send the request to. This possibility is useful, for instance, in the following cases:

1. An O-MI “proxy” node is used as a unified message router inside a company network (or a Smart House or similar) that takes care of forwarding and receiving all messages through a firewall. In this case, all messages are sent to the “proxy” node that then takes care of forwarding the messages to the recipient nodes. In this case, the “proxy” node also needs to take care of routing responses back to the original sender O-MI node.
2. An event needs to be sent by a write request to many different destination nodes, but the sending O-MI node does not have message persistence (buffering), callback, or similar needed capacities for ensuring that the request reaches its destination.
3. For some reason, we need information about the same product (or some other item) from a predefined and known list of O-MI nodes and the requesting O-MI node cannot manage all the needed read requests itself.

Correspondingly, an O-MI node that receives a request with explicitly defined destination nodes SHALL respond with *result* elements that include *nodeLists* that indicate from which O-MI node(s) the results come.

As seen from the example cases given above, the possibility to explicitly specify a list of destination nodes is mainly related to the use of a “proxy” O-MI node that provides some functionality that the originating O-MI node does not implement. However, it is also imaginable that the *nodeList* element may be used for other purposes, depending on application requirements.

Information	**Description**
nodeList	List of recipient O-MI nodes. The requesting O-MI node MAY include a list of recipient nodes for the request. If included, the O-MI node receiving the request SHALL send it only to those nodes. If not included, the O-MI node receiving the request is the only recipient node.

6 Error Handling

Error situations and faults can occur on several different levels in O-MI communication and usage.

Errors concerned with the communication between an O-MI requester and an O-MI request handler should be dealt with on the communication level. This is explained in Section 6.1.

Information about errors that affect the handing of O-MI requests should be included in the O-MI responses. These are defined in Section 6.2.

6.1 Communication-Level Errors

Communication-level errors are handled by the communication layer. This is typically an HTTP layer (both if pure HTTP communication or Web Service communication is used) so the O-MI nodes SHOULD use HTTP error messages and codes. In addition, the O-MI nodes MAY use other error messages typical for the communication protocol.

6.2 O-MI Content-Level Errors

Every *result* element of an O-MI response SHALL have a *return* child element with a *returnCode* attribute. This SHALL have HTTP Status Codes as defined in IETF RFC 2616 as values. The *return* element MAY also contain other attributes for informative purposes, such as providing more detailed or application-specific information, but their names and usage are not covered by this standard.

For HTTP Status Codes, see, for instance: www.w3.org/Protocols/HTTP/HTRESP.html or www.w3.org/Protocols/rfc2616/rfc2616-sec10.html.

A O-MI XSD Schema (Normative)

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Kary Främling (Aalto University) -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
xmlns="http://www.opengroup.org/xsd/omi/2.0/"
targetNamespace="http://www.opengroup.org/xsd/omi/2.0/"
elementFormDefault="qualified" attributeFormDefault="unqualified"
version="2.0">
  <xs:element name="omiEnvelope" type="OmiEnvelopeType"/>
  <xs:complexType name="OmiEnvelopeType">
    <xs:annotation>
      <xs:documentation>Root element of Open Messaging
Interface.</xs:documentation>
    </xs:annotation>
    <xs:sequence>
      <xs:choice maxOccurs="unbounded">
        <xs:element name="read" type="ReadRequestType"/>
        <xs:element name="write" type="WriteRequestType"/>
        <xs:element name="response" type="ResponseListType"/>
        <xs:element name="cancel" type="CancelRequestType"/>
        <xs:element name="call" type="CallRequestType"/>
        <xs:element name="delete" type="DeleteRequestType"/>
      </xs:choice>
    </xs:sequence>
    <xs:attribute name="version" type="xs:string" use="required">
      <xs:annotation>
        <xs:documentation>Open messaging interface schema version that the
message complies with.</xs:documentation>
      </xs:annotation>
    </xs:attribute>
    <xs:attribute name="ttl" use="required">
      <xs:annotation>
        <xs:documentation>Time-to-live in seconds. "-1" signifies
"forever"</xs:documentation>
      </xs:annotation>
      <xs:simpleType>
        <xs:union memberTypes="PositiveDouble DoubleMinusOne"/>
      </xs:simpleType>
    </xs:attribute>
    <xs:attribute name="authorization" type="xs:string" use="optional">
      <xs:annotation>
        <xs:documentation>Authorization header contents according to IETF RFC
7235, §4.2: Authorization. Since O-MI v2.0.</xs:documentation>
      </xs:annotation>
    </xs:attribute>
    <xs:anyAttribute processContents="lax">
      <xs:annotation>
        <xs:documentation>Proprietary or extended attributes. Since O-MI
v2.0.</xs:documentation>
      </xs:annotation>
    </xs:anyAttribute>
  </xs:complexType>
  <xs:simpleType name="PositiveDouble">
    <xs:restriction base="xs:double">
      <xs:minInclusive value="0"/>
    </xs:restriction>
```

```
  </xs:simpleType>
  <xs:simpleType name="DoubleMinusOne">
    <xs:restriction base="xs:double">
      <xs:minInclusive value="-1"/>
      <xs:maxInclusive value="-1"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="DoubleMinusTwo">
    <xs:restriction base="xs:double">
      <xs:minInclusive value="-2"/>
      <xs:maxInclusive value="-2"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="MsgType" mixed="true">
    <xs:annotation>
      <xs:documentation>Payload for the protocol</xs:documentation>
    </xs:annotation>
    <xs:sequence>
      <xs:any namespace="##other" processContents="strict" minOccurs="0"
maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="RequestBaseType" abstract="true">
    <xs:annotation>
      <xs:documentation>Base type for "read", "write", "call" and "delete"
requests.</xs:documentation>
    </xs:annotation>
    <xs:sequence>
      <xs:element name="nodeList" type="NodesType" minOccurs="0"/>
      <xs:choice>
      <xs:element name="requestID" type="xs:string" minOccurs="0"
maxOccurs="unbounded">
      <xs:annotation>
        <xs:documentation>"requestID" is only included when "polling" for data
that corresponds to an earlier subscription that returned the corresponding
Id.</xs:documentation>
      </xs:annotation>
      </xs:element>
      <xs:element name="msg" type="MsgType" minOccurs="0"/>
      </xs:choice>
    </xs:sequence>
    <xs:attribute name="callback" type="xs:anyURI" use="optional">
      <xs:annotation>
        <xs:documentation>URI of callback O-MI node where responses should be
sent.</xs:documentation>
      </xs:annotation>
    </xs:attribute>
    <xs:attribute name="msgformat" type="SchemaIDType" use="optional"/>
    <xs:attribute name="targetType" type="TargetTypeType" use="optional"
default="node"/>
  </xs:complexType>
  <xs:simpleType name="TargetTypeType">
    <xs:annotation>
      <xs:documentation>Using "device" indicates that if the message "target
object" is some kind of device connected to a node, then try to get the
requested value from the "device".</xs:documentation>
    </xs:annotation>
    <xs:restriction base="xs:string">
      <xs:enumeration value="device"/>
      <xs:enumeration value="node"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="ReadRequestType">
```

```
    <xs:complexContent>
      <xs:extension base="RequestBaseType">
        <xs:attribute name="interval" use="optional">
          <xs:annotation>
            <xs:documentation>
              If an "interval" attribute is included, it indicates that this is
a subscription request that uses the given interval.
              Values 0 (as often as data can be read from the target),
              -1 (very time the subscribed target publishes/pushes a new value)
              and -2 (every time the subscribed target (typically a PEID or
device) connects to an O-MI node)
              have special significations.
            </xs:documentation>
          </xs:annotation>
          <xs:simpleType>
            <xs:union memberTypes="PositiveDouble DoubleMinusOne
DoubleMinusTwo"/>
          </xs:simpleType>
        </xs:attribute>
        <xs:attribute name="oldest" type="xs:positiveInteger" use="optional">
          <xs:annotation>
            <xs:documentation>Retrieve the oldest available number of
historical data available.</xs:documentation>
          </xs:annotation>
        </xs:attribute>
        <xs:attribute name="begin" type="xs:dateTime" use="optional">
          <xs:annotation>
            <xs:documentation>Retrieve data from this begin
date.</xs:documentation>
          </xs:annotation>
        </xs:attribute>
        <xs:attribute name="end" type="xs:dateTime" use="optional">
          <xs:annotation>
            <xs:documentation>Retrieve data until this end
date.</xs:documentation>
          </xs:annotation>
        </xs:attribute>
        <xs:attribute name="newest" type="xs:positiveInteger" use="optional">
          <xs:annotation>
            <xs:documentation>Retrieve the newest available number of
historical data available</xs:documentation>
          </xs:annotation>
        </xs:attribute>
        <xs:attribute name="all" type="xs:boolean" use="optional">
          <xs:annotation>
            <xs:documentation>Retrieve all available values. all="true"
overrides other begin/end/oldest/newest attributes. From Version 2.0
onwards.</xs:documentation>
          </xs:annotation>
        </xs:attribute>
        <xs:attribute name="maxlevels" type="xs:positiveInteger"
use="optional">
          <xs:annotation>
            <xs:documentation>The maximum number of levels (Object/InfoItem) to
return from an O-DF hierarchy, for instance. The leaf Object(s) of the read
request are counted as level zero, so "maxlevels=1" signifies that the only the
next level of Objects/InfoItems should be returned. From Version 2.0
onwards.</xs:documentation>
          </xs:annotation>
        </xs:attribute>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
```

```
  <xs:simpleType name="SchemaIDType">
    <xs:annotation>
      <xs:documentation>Some "predefined" identifier for the format/semantics
of "value"; e.g., "csv", "obix.xsd".</xs:documentation>
    </xs:annotation>
    <xs:restriction base="xs:string"/>
  </xs:simpleType>
  <xs:complexType name="WriteRequestType">
    <xs:complexContent>
      <xs:extension base="RequestBaseType"/>
    </xs:complexContent>
  </xs:complexType>
  <xs:complexType name="ResponseListType">
    <xs:annotation>
      <xs:documentation>List of results.</xs:documentation>
    </xs:annotation>
    <xs:sequence>
      <xs:element name="result" type="RequestResultType"
maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:complexType name="CallRequestType">
    <xs:annotation>
      <xs:documentation>Call request type.</xs:documentation>
    </xs:annotation>
    <xs:complexContent>
      <xs:extension base="RequestBaseType"/>
    </xs:complexContent>
  </xs:complexType>
  <xs:complexType name="DeleteRequestType">
    <xs:annotation>
      <xs:documentation>Delete request type.</xs:documentation>
    </xs:annotation>
    <xs:complexContent>
      <xs:extension base="RequestBaseType"/>
    </xs:complexContent>
  </xs:complexType>
  <xs:complexType name="RequestResultType">
    <xs:annotation>
      <xs:documentation>Result of a request.</xs:documentation>
    </xs:annotation>
    <xs:sequence>
        <xs:element name="return" type="ReturnType" minOccurs="1">
          <xs:annotation>
            <xs:documentation>Indicates success/failure of
request.</xs:documentation>
          </xs:annotation>
        </xs:element>
        <xs:element name="requestID" type="IdType" minOccurs="0"
maxOccurs="unbounded ">
          <xs:annotation>
            <xs:documentation>not needed if this is a response to a
"immediate"/"non-callback" request; i.e., just status reply with "return" or
response with "value".</xs:documentation>
          </xs:annotation>
        </xs:element>
      <xs:element name="msg" type="MsgType" minOccurs="0">
        <xs:annotation>
          <xs:documentation>response value. Completely free-format. Present if
successful.</xs:documentation>
        </xs:annotation>
      </xs:element>
      <xs:element name="nodeList" type="NodesType" minOccurs="0">
```

```
        <xs:annotation>
          <xs:documentation>may be used to indicate from what node(s) the
result comes</xs:documentation>
        </xs:annotation>
      </xs:element>
      <xs:element ref="omiEnvelope" minOccurs="0">
        <xs:annotation>
          <xs:documentation>Present if the sender wants to submit a new request
to the receiver. This is useful for engaging direct dialogs" while the
connection is alive. It is also especially useful for communicating with
firewall/NAT-protected nodes.</xs:documentation>
        </xs:annotation>
      </xs:element>
    </xs:sequence>
    <xs:attribute name="msgformat" type="SchemaIDType" use="optional"/>
    <xs:attribute name="targetType" type="TargetTypeType" use="optional"
default="node"/>
  </xs:complexType>
  <xs:complexType name="ReturnType">
    <xs:annotation>
      <xs:documentation>Return status of request. Use HTTP codes / descriptions
when applicable.</xs:documentation>
    </xs:annotation>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="returnCode" use="required">
          <xs:annotation>
            <xs:documentation>Use HTTP codes when
applicable.</xs:documentation>
          </xs:annotation>
          <xs:simpleType>
            <xs:restriction base="xs:token">
              <xs:pattern value="2[0-9]{2}|4[0-9]{2}|5[0-9]{2}"/>
            </xs:restriction>
          </xs:simpleType>
        </xs:attribute>
        <xs:attribute name="description" type="xs:string" use="optional"/>
        <xs:anyAttribute/ processContents="lax">
          <xs:annotation>
            <xs:documentation>Proprietary or extended attributes. Since O-MI
v2.0.</xs:documentation>
          </xs:annotation>
        </xs:anyAttribute>>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
  <xs:complexType name="NodesType">
    <xs:annotation>
      <xs:documentation>The nodesType is used anywhere in the schema where
lists of nodes can appear. </xs:documentation>
    </xs:annotation>
    <xs:sequence>
      <xs:element name="node" type="xs:anyURI" minOccurs="1"
maxOccurs="unbounded">
        <xs:annotation>
          <xs:documentation>Defines the URL/URI of an O-MI node. The node may
be located using other means than this URL field, like discovery and routing
functionality.</xs:documentation>
        </xs:annotation>
      </xs:element>
    </xs:sequence>
    <xs:attribute name="type" type="xs:string" use="optional">
      <xs:annotation>
```

```
        <xs:documentation>String indicating what format is being used for
"node" addresses. By default, a URL. </xs:documentation>
      </xs:annotation>
    </xs:attribute>
  </xs:complexType>
  <xs:complexType name="IdType">
    <xs:annotation>
      <xs:documentation>Some kind of identifier with optional "format"
attribute for indicating what kind of identifier is used. </xs:documentation>
    </xs:annotation>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="format" type="xs:string" use="optional">
          <xs:annotation>
            <xs:documentation>Use for indicating what kind of
identifier.</xs:documentation>
          </xs:annotation>
        </xs:attribute>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
  <xs:complexType name="CancelRequestType">
    <xs:sequence>
      <xs:element name="nodeList" type="NodesType" minOccurs="0"/>
      <xs:element name="requestID" type="IdType" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

B Example Messages

This appendix shows example messages for some basic cases. More examples are normally available at the website(s) where this standard is published.

All examples have been validated against the XML schema using the UNIX `xmllint` utility. For validating against both O-MI and O-DF schemas, the following schema file was created:

```
<?xml version="1.0" encoding="UTF-8"?>
<schema elementFormDefault="qualified" xmlns="http://www.w3.org/2001/XMLSchema">
    <import namespace="http://www.opengroup.org/xsd/omi/2.0/" schemaLocation="omi.xsd"/>
    <import namespace="http://www.opengroup.org/xsd/odf/2.0/" schemaLocation="odf.xsd"/>
    <import namespace="http://obix.org/ns/schema/1.0" schemaLocation="obix.xsd"/>
</schema>
```

This was necessary because `xmllint` does not support more than one schema file. The file was called `OMI_ODF.xsd` and the command that was executed for validation was:

```
xmllint --noout --schema OMI_ODF.xsd <XML file>
```

Example 5: Read Request Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of a simple read message using ODF payload. Namespace based
validation included. -->
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
    <read msgformat="omi.xsd">
      <!-- Here could be a list of destination nodes if the message can't
      be sent directly to the destination node(s). -->
      <msg>
        <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
         xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd">
           <Object>
              <id>SmartFridge22334411</id>
              <InfoItem name="PowerConsumption">
              </InfoItem>
            </Object>
        </Objects>
      </msg>
    </read>
</omiEnvelope>
```

Example 6: Read Response

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of a simple reply message using ODF payload. Namespace based
validation included. -->
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <response>
    <result msgformat="odf">
      <return returnCode="200"></return>
```

```
      <msg>
        <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
         xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd">
          <Object>
            <id>SmartFridge22334411</id>
            <!-- Only most recent value, bogus timestamp. -->
            <InfoItem name="PowerConsumption">
            <value type="xs:int" unixTime="5453563">43</value>
            </InfoItem>
          </Object>
        </Objects>
      </msg>
    </result>
  </response>
</omiEnvelope>
```

Example 7: Read Response Metadata Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of a response message to metadata request using OMI_d payload.
Namespace based validation included. -->
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <response>
    <result msgformat="odf">
      <return returnCode="200"></return>
      <msg>
        <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
              xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/
odf.xsd">
          <Object>
            <id>SmartFridge22334411</id>
            <InfoItem name="PowerConsumption">
              <MetaData>
              <InfoItem name="format"><value
type="xs:string">xs:double</value></InfoItem>
              <InfoItem name="latency"><value
type="xs:int">5</value></InfoItem>
              <InfoItem name="readable"><value
type="xs:boolean">true</value></InfoItem>
              <InfoItem name="writable"><value
type="xs:boolean">false</value></InfoItem>
              <InfoItem name="unit"><value
type="xs:string">Watts</value></InfoItem>
              <InfoItem name="accuracy"><value
type="xs:double">1</value></InfoItem>
              </MetaData>
            </InfoItem>
          </Object>
        </Objects>
      </msg>
    </result>
  </response>
</omiEnvelope>
```

Example 8: Multiple Payload Response Example

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of a response message using several different payloads. -->
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <response>
    <result msgformat="obix">
    <return returnCode="200"></return>
      <msg>
        <obj xsi:schemaLocation="http://obix.org/ns/schema/1.0 obix.xsd"
                xmlns="http://obix.org/ns/schema/1.0"
                href="http://myhome/thermostat" >
          <real name="spaceTemp" unit="obix:units/fahrenheit" val="67.2"/>
          <real name="setpoint" unit="obix:units/fahrenheit" val="72.0"/>
          <bool name="furnaceOn" val="true"/>
        </obj>
      </msg>
    </result>
    <result msgformat="CSV">
    <return returnCode="200"></return>
    <msg>11,22,33
          44,55,66</msg>
    </result>
    <result msgformat="odf">
    <return returnCode="200"></return>
    <msg>
    <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
                xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/
odf.xsd">
        <Object>
        <id>SmartFridge22334411</id>
          <InfoItem name="PowerConsumption"><value>43</value></InfoItem>
        </Object>
      </Objects>
    </msg>
    </result>
  </response>
</omiEnvelope>
```

Example 9: Write Request

```
<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <write msgformat="odf" targetType="device">
    <msg>
      <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
        xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd">
        <Object>
        <id>SmartFridge22334411</id>
          <InfoItem
name="FridgeTemperatureSetpoint"><value>3.5</value></InfoItem>
          <InfoItem name="FreezerTemperatureSetpoint"><value>-
20.0</value></InfoItem>
        </Object>
      </Objects>
    </msg>
  </write>
</omiEnvelope>
```

Example 10: Error Response

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of an error response. -->
```

```
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <response>
    <result>
    <return returnCode="404"></return>
    </result>
  </response>
</omiEnvelope>
```

Example 11: Cancel Request

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of O-MI cancel message for cancelling two requests with different
request IDs. -->
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <cancel>
    <requestID>REQ0011212121212</requestID>
    <requestID>REQ0011212121213</requestID>
  </cancel>
</omiEnvelope>
```

C Example Messages for Method Publication and Call

The example messages shown in this appendix are based on specification work for publishing a service that returns a list of charging stations that are close to a given destination and available at the given time and for the given duration.

The information model used in these examples is mainly based on schema.org specifications and data types.

Example 12: Publish Method by *write* Request

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Example of publishing the data model for an electrical vehicle
charging service using write method. -->
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <write msgformat="odf">
    <msg>
     <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd"
version="2.0">
        <Object type="EVchargingService"> <!-- Generic type for EV charging
services -->
            <id>EVchargingServiceHelsinki1</id> <!-- id of this specific EV
charging service instance -->
            <InfoItem name="FindEV_charging">
          <MetaData>
            <InfoItem
name="odf:InfoItemType"><value>"odf:Method"</value></InfoItem>
            <InfoItem name="odf:Parameter">
              <!-- "x": "odf:Parameter" to be documented as reserved name. -->
              <value >
                <Objects>
                  <Object type="FindEV_chargingRequest">
                    <id idType="requestID">ID_PLACEHOLDER</id>
                    <InfoItem name="MaxPerimeter" type="ref-to-some-type-
specifying-perimeter"><MetaData><InfoItem
name="unit"><value>km</value></InfoItem></MetaData></InfoItem>
                    <InfoItem name="When"><value
type="xs:date">VALUE_PLACEHOLDER</value></InfoItem>
                    <InfoItem name="NeededCharge">
                      <MetaData>
                        <InfoItem name="unit"><value>Ah</value></InfoItem>
                      </MetaData>
                      <value>VALUE_PLACEHOLDER</value>
                    </InfoItem>
                    <Object type="https://schema.org/GeoCoordinates">
                      <id>Destination</id>
                      <InfoItem name="o-def-name">
                          <value>Environment_Measure;UNECERec20:[needs-to-be-
found]</value>
                      </InfoItem>
```

```
                      <InfoItem
name="latitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                      <InfoItem
name="longitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                    </Object>
                  </Object>
                </Objects>
              </value>
            </InfoItem>
            <InfoItem name="odf:ReturnType">
              <value>
                <Objects>
                  <Object type="EVchargingReservation">
                    <id>ID_PLACEHOLDER</id>
                    <InfoItem name="ValidUntil"><value
type="xs:date">PLACE_PLACEHOLDER</value></InfoItem>
                    <InfoItem name="ReservationCode"><value
type="xs:int">PLACE_PLACEHOLDER</value></InfoItem>
                    <Object type="https://schema.org/GeoCoordinates">
                      <id>Location</id>
                      <InfoItem
name="latitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                      <InfoItem
name="longitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                    </Object>
                   </Object>
                </Objects>
              </value>
            </InfoItem>
          </MetaData>
          <value>odf:Method</value>
        </InfoItem>
         </Object>
    </Objects>
       </msg>
    </write>
</omiEnvelope>
```

Example 13: Read Request and Response for Method

```
<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
    <read msgformat="odf">
        <msg>
            <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
            xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd"
            version="2.0">
                <Object type="EVchargingService">
                    <id>EVchargingServiceHelsinki1</id>
                    <InfoItem name="FindEV_charging"/>
                </Object>
            </Objects>
        </msg>
    </read>
</omiEnvelope>

<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
```

```
version="2.0" ttl="10">
    <response>
        <result msgformat="odf">
            <return returnCode="200"></return>
            <msg>
      <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
      xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd"
      version="2.0">
        <Object type="EVchargingService">
          <id>EVchargingServiceHelsinki1</id>
          <InfoItem name="FindEV_charging">
            <value>odf:Method</value>
          </InfoItem>
        </Object>
      </Objects>
            </msg>
        </result>
    </response>
</omiEnvelope>
```

Example 14: Read Request and Response for Method MetaData

```
<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
    <read msgformat="odf">
        <msg>
            <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd">
                <Object type="EVchargingService">
                    <id>EVchargingServiceHelsinki1</id>
                    <InfoItem name="FindEV_charging"><MetaData/></InfoItem>
                </Object>
            </Objects>
        </msg>
    </read>
</omiEnvelope>

<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
    <response>
        <result msgformat="odf">
            <return returnCode="200"></return>
            <msg>
      <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd"
version="2.0">
        <Object type="EVchargingService">
          <id>EVchargingServiceHelsinki1</id>
          <InfoItem name="FindEV_charging">
            <MetaData>
              <InfoItem
name="InfoItemType"><value>"odf:Method"</value></InfoItem>
              <InfoItem name="odf:Parameter"><value >
                <Objects>
                  <Object type="FindEV_chargingRequest">
                    <id idType="requestID">ID_PLACEHOLDER</id>
```

```
                    <InfoItem name="MaxPerimeter"><MetaData><InfoItem
name="unit"><value>km</value></InfoItem></MetaData><value>VALUE_PLACEHOLDER</va
lue></InfoItem>
                    <InfoItem name="When"><value
type="xs:date">VALUE_PLACEHOLDER</value></InfoItem>
                    <InfoItem name="NeededCharge">
                      <MetaData>
                        <InfoItem name="unit"><value>Ah</value></InfoItem>
                      </MetaData>
                      <value>VALUE_PLACEHOLDER</value>
                    </InfoItem>
                    <Object type="https://schema.org/GeoCoordinates">
                      <id>Destination</id>
                      <InfoItem
name="latitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                      <InfoItem
name="longitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                    </Object>
                  </Object>
                </Objects>
              </value></InfoItem>
              <InfoItem name="odf:ReturnType"><value>
                <Objects>
                  <Object type="EVchargingReservation">
                    <id>ID_PLACEHOLDER</id>
                    <InfoItem name="ValidUntil"><value
type="xs:date">PLACE_PLACEHOLDER</value></InfoItem>
                    <InfoItem name="ReservationCode"><value
type="xs:int">PLACE_PLACEHOLDER</value></InfoItem>
                    <Object type="https://schema.org/GeoCoordinates">
                      <id>Location</id>
                      <InfoItem
name="latitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                      <InfoItem
name="longitude"><value>VALUE_PLACEHOLDER</value></InfoItem>
                    </Object>
                  </Object>
                </Objects>
              </value></InfoItem>
            </MetaData>
          </InfoItem>
        </Object>
      </Objects>
            </msg>
        </result>
    </response>
</omiEnvelope>
```


Example 15: Call Request and Response for Method


```
<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
    <call msgformat="odf">
        <msg>
            <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd">
                <Object type="EVchargingService">
                    <id>EVchargingServiceHelsinki1</id>
                    <InfoItem name="FindEV_charging">
            <value type="odf:Parameter">
```

```
              <Objects>
                <Object type="O-MI_parameter">
                  <id >EVchargingRequest9876543</id>
                  <InfoItem name="MaxPerimeter"><value>0.5</value></InfoItem>
                  <InfoItem name="When"><value>"2016-11-
21T15:24:29.271+02:00"</value></InfoItem>
                  <InfoItem name="NeededCharge"><value>92</value></InfoItem>
                  <Object type="https://schema.org/GeoCoordinates">
                    <id>Destination</id>
                    <InfoItem
name="latitude"><value>60.186669</value></InfoItem>
                    <InfoItem
name="longitude"><value>24.827682</value></InfoItem>
                  </Object>
                </Object>
              </Objects>
            </value>
          </InfoItem>
                </Object>
            </Objects>
        </msg>
    </call>
</omiEnvelope>

<?xml version="1.0" encoding="UTF-8"?>
<omiEnvelope xmlns="http://www.opengroup.org/xsd/omi/2.0/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.opengroup.org/xsd/omi/2.0/ omi.xsd"
version="2.0" ttl="10">
  <response>
    <result msgformat="odf">
      <!-- <return returnCode="200"></return> -->
      <requestID>EVchargingRequest9876543</requestID>
      <msg>
        <Objects xmlns="http://www.opengroup.org/xsd/odf/2.0/"
xsi:schemaLocation="http://www.opengroup.org/xsd/odf/2.0/ odf.xsd">
          <Object type="EVchargingReservation">
            <id>EVchargingReservation3421432432</id>
            <InfoItem name="ValidUntil">
              <value type="xs:date">"2016-11-21T15:24:29.271+02:00"</value>
            </InfoItem>
            <InfoItem name="ReservationCode">
              <value type="xs:int">"5435434"</value>
            </InfoItem>
            <Object type="https://schema.org/GeoCoordinates">
              <id>Location</id>
              <InfoItem name="latitude">
                <value>60.186669</value>
              </InfoItem>
              <InfoItem name="longitude">
                <value>24.827682</value>
              </InfoItem>
            </Object>
          </Object>
        </Objects>
      </msg>
    </result>
  </response>
</omiEnvelope>
```

D JSON Mapping and Examples

Many XML generation and manipulation tools can convert XML messages into JavaScript Object Notation (JSON), and *vice versa*. There are also many frameworks that can do the needed conversions in “real time”. The following code shows Example 8 converted into JSON by an online tool (in this case at URL http://www.utilities-online.info/xmltojson/#.XBuT6y2B1TY).

```
{
  "omiEnvelope": {
    "-xmlns": "http://www.opengroup.org/xsd/omi/2.0/",
    "-xmlns:xsi": "http://www.w3.org/2001/XMLSchema-instance",
    "-xsi:schemaLocation": "http://www.opengroup.org/xsd/omi/2.0/ omi.xsd",
    "-version": "2.0",
    "-ttl": "10",
    "write": {
      "-msgformat": "odf",
      "-targetType": "device",
      "msg": {
        "Objects": {
          "-xmlns": "http://www.opengroup.org/xsd/odf/2.0/",
          "-xsi:schemaLocation": "http://www.opengroup.org/xsd/odf/2.0/
odf.xsd",
          "Object": {
            "id": "SmartFridge22334411",
            "InfoItem": [
              {
                "-name": "FridgeTemperatureSetpoint",
                "value": "3.5"
              },
              {
                "-name": "FreezerTemperatureSetpoint",
                "value": "-20.0"
              }
            ]
          }
        }
      }
    }
  }
}
```

Another conversion tool (https://codebeautify.org/xmltojson) produces the following:

```
{
  "omiEnvelope": {
    "write": {
      "msg": {
        "Objects": {
          "Object": {
            "id": "SmartFridge22334411",
            "InfoItem": [
              {
                "value": "3.5",
                "_name": "FridgeTemperatureSetpoint"
              },
              {
```

```
                "value": "-20.0",
                "_name": "FreezerTemperatureSetpoint"
              }
            ]
          },
          "_xmlns": "http://www.opengroup.org/xsd/odf/2.0/",
          "_xsi:schemaLocation": "http://www.opengroup.org/xsd/odf/2.0/
odf.xsd"
        }
      },
      "_msgformat": "odf",
      "_targetType": "device"
    },
    "_xmlns": "http://www.opengroup.org/xsd/omi/2.0/",
    "_xmlns:xsi": "http://www.w3.org/2001/XMLSchema-instance",
    "_xsi:schemaLocation": "http://www.opengroup.org/xsd/omi/2.0/ omi.xsd",
    "_version": "2.0",
    "_ttl": "10"
  }
}
```

Abbreviations

CSV	Comma-Separated Value
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IoT	Internet of Things
JMS	Java Message Service
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
NAT	Network Address Translation
oBIX	Open Building Information eXchange
O-DF	Open Data Format
O-MI	Open Messaging Interface
PEID	Product Embedded Information Device
QLM	Quantum Lifecycle Management
REST	REpresentational State Transfer
RFID	Radio-Frequency Identification
SMTP	Simple Mail Transfer Protocol
SOAP	Simple Object Access Protocol
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTL	Time-to-Live
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
WSDL	Web Services Description Language
WWW	World-Wide Web
XML	EXtensible Markup Language

Index

call request..... 13, 20
delete request..... 14, 21
HTTP..... 6
HTTPS..... 6
Internet of Things 1
JMS..... 6
JSON 42
MQTT..... 6
Observer 6
O-DF..... 2
O-MI requests 10
one-time read request 18
Publish/Subscribe..... 6
read request 10
subscription request..... 19
TLS..... 7
WebSocket 6, 7
write request..... 12, 20
WSDL/SOAP 6
XSD schema..... 26